ML Models Hub

A Modular Streamlit Application

Rachel Monique Tumulak

Algonquin College

Problem

- Machine learning projects are usually built in notebooks
- Notebooks are not user-friendly for real interaction
- Hard to reuse models for real inputs
- No structured interface for prediction and analysis

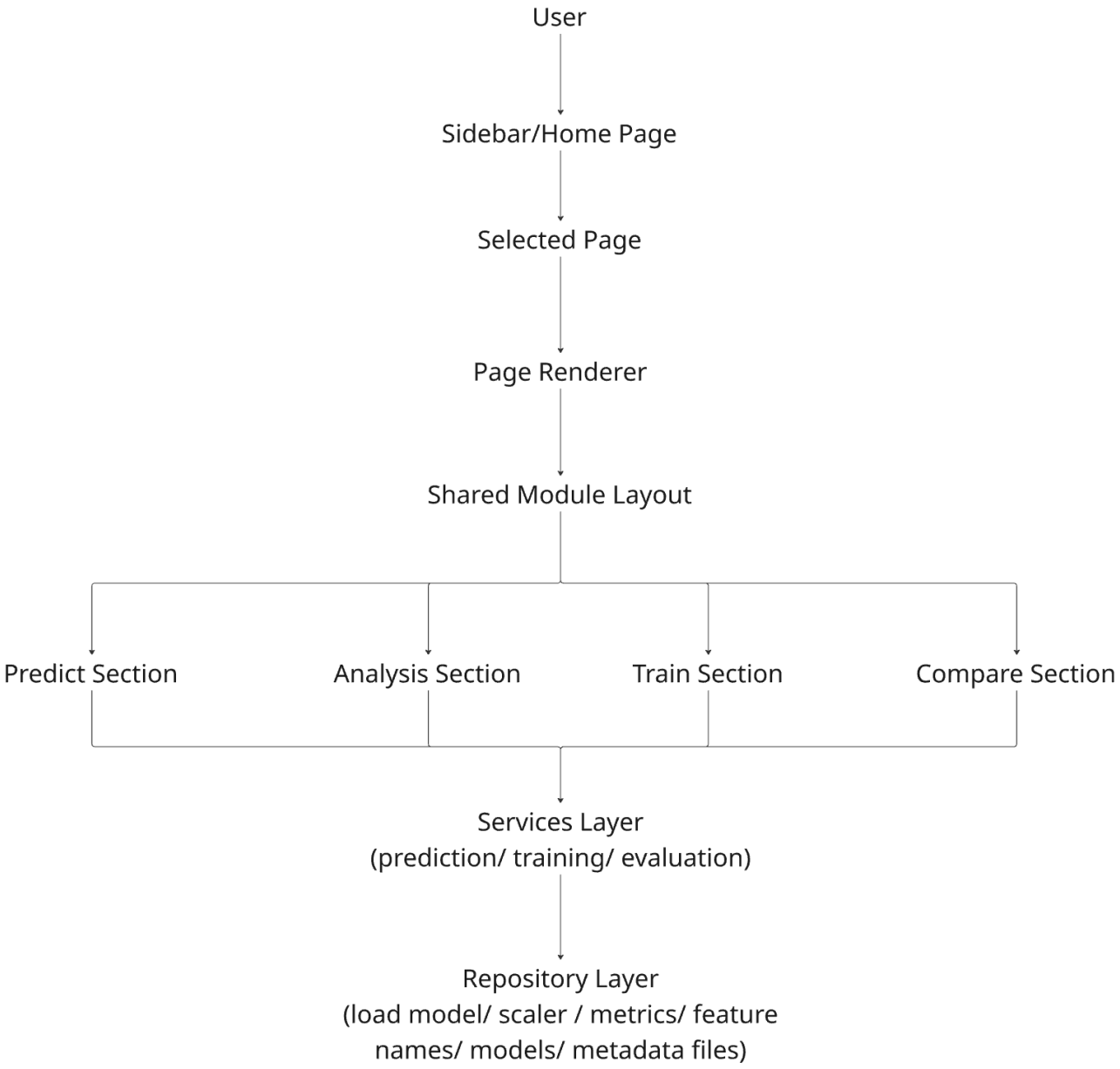
Solution

- Built a Streamlit application called ML Models Hub
- Combines multiple ML workflows in one system
- Converts notebook models into an interactive app
- Provides prediction, analysis, tuning, and comparison

Implemented Modules

- Loan Eligibility (Classification)
- Real Estate Price Prediction (Regression)
- Student Admission (Neural Network)
- Customer Segmentation (Clustering)

System Architecture



Architecture Overview

- Sidebar navigation selects module
- Shared layout across all modules
- Four sections:
  - Predict
  - Model Analysis
  - Tune / Custom Model
  - Compare
- Services handle ML logic
- Repository loads models and artifacts

System Workflow

- Models developed in Jupyter notebooks
- Models saved as serialized files
- App loads models dynamically
- User inputs processed through services
- Results displayed in UI

Home Page

- Displays overview of the system
- Shows number of modules and models
- Lists available workflows

Deploy

ML Models Hub

A modular machine learning hub for prediction, analysis, tuning, and comparison.

Modules	Default Models	Total Models	Workflows
4	6	11	4

Platform Overview

This application brings together four machine learning workflows in one place. Each module is organized around a consistent structure so users can explore default models, review analysis, tune model settings, and compare results.

Available Modules

module	task	focus
Loan Eligibility	Classification	Predict loan approval outcomes from applicant informatio
Real Estate	Regression	Estimate property values and examine feature relationship
Student Admission	Neural Network Prediction	Model admission likelihood using academic profile inputs.
Customer Segmentation	Clustering	Group customers into segments based on shared characte

How the Hub Is Organized

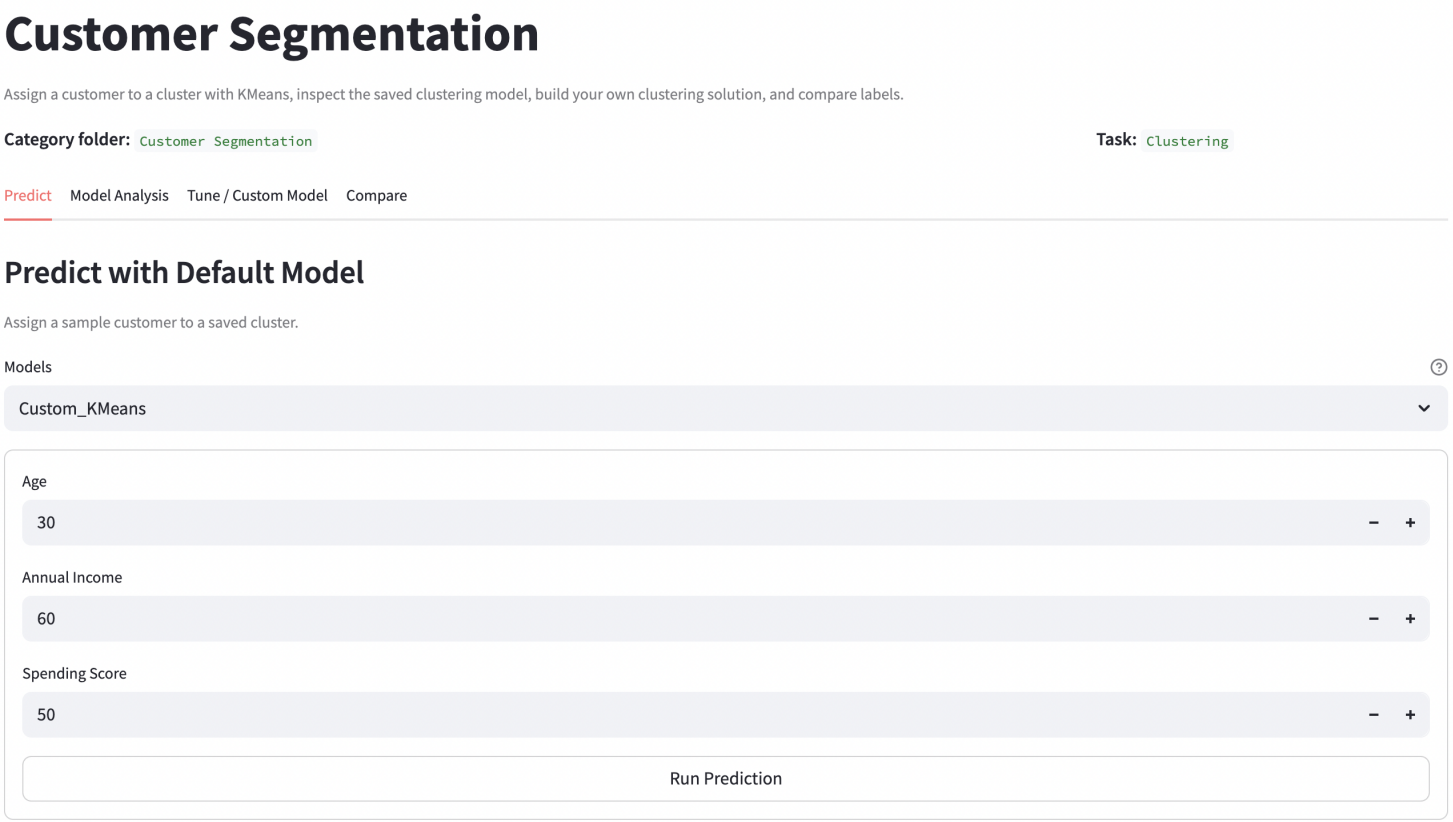
Use the sidebar to open any module page.

Each module is designed around:

- prediction with default models
- model analysis
- parameter tuning
- result comparison

Example: Customer Segmentation

- Uses K-Means clustering
- Inputs:
  - Age
  - Income
  - Spending Score
- Outputs cluster assignment



Predict Section

- User inputs feature values
- Default model generates prediction
- Immediate feedback to user

Annual Income

60 − +

Spending Score

50 − +

Run Prediction

Assigned cluster: 3

Your Position in the Cluster Map

Cluster centers

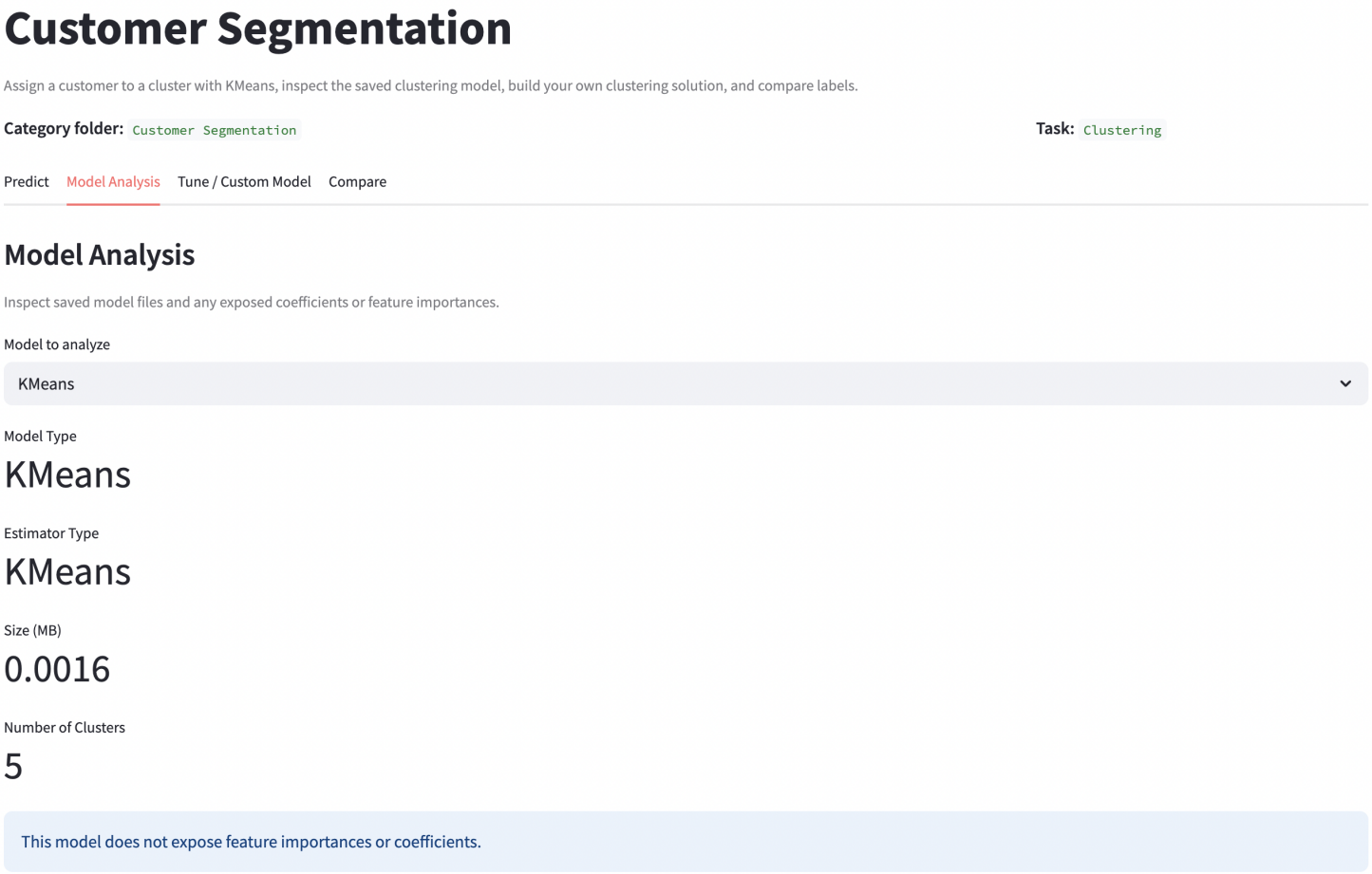
0	1	2
32.6923	86.5385	82.1282
24.8	41.46	63.7
53.7115	54.4231	48.7308
44.1429	25.1429	19.5238
40.3947	87	18.6316

Processed feature vector

	Age	Annual_Income	Spending_Score
0	30	60	50

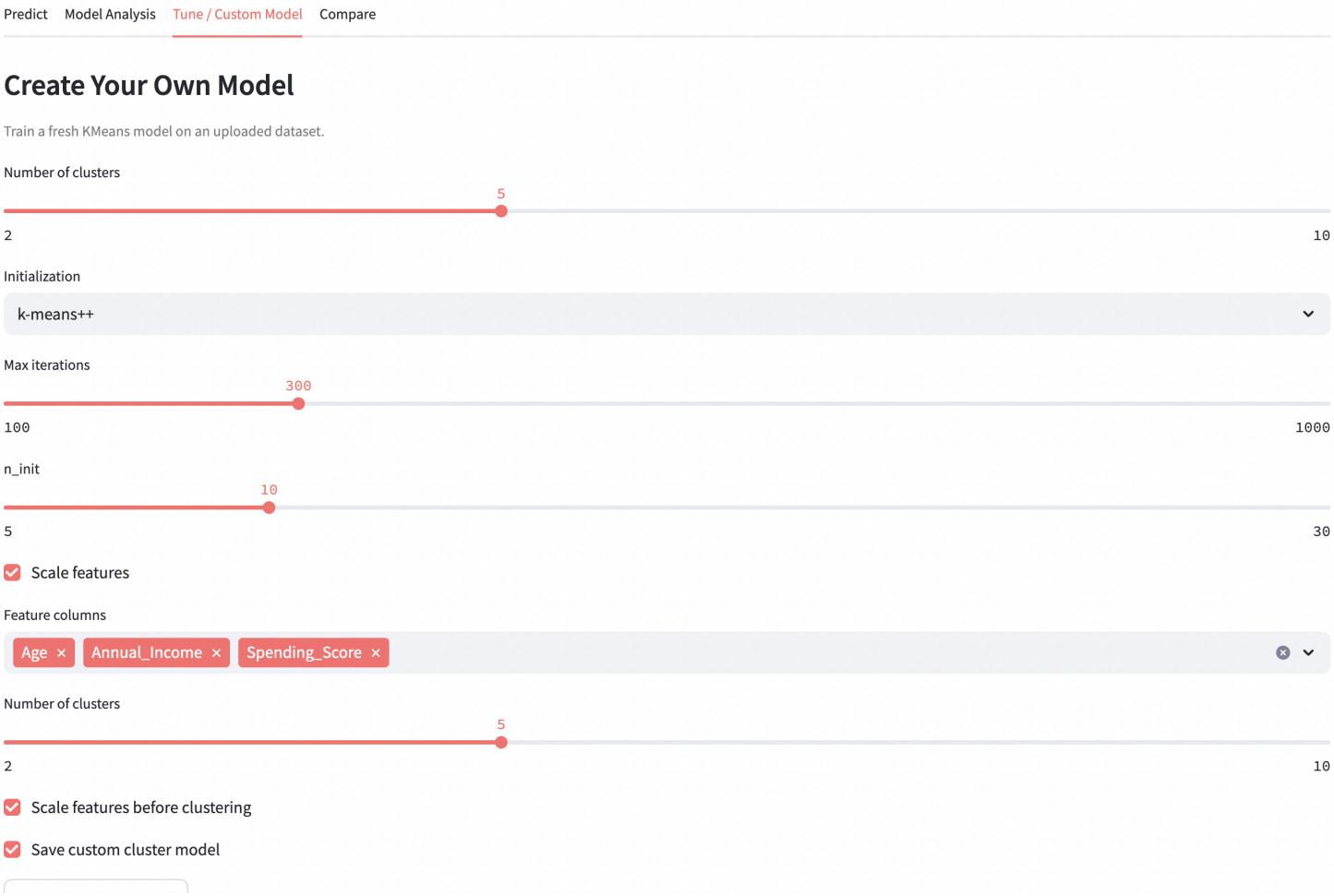
Model Analysis

- Displays model information
- Shows metrics and interpretation
- Helps understand model behavior



Tune / Custom Model

- Adjust model parameters
- Retrain model inside app
- Uses built-in dataset



Model Comparison

- Compare default vs tuned model
- Evaluate performance differences
- Supports better decision-making

Customer Segmentation

Assign a customer to a cluster with KMeans, inspect the saved clustering model, build your own clustering solution, and compare labels.

Category folder: `Customer Segmentation` **Task:** `Clustering`

Predict Model Analysis Tune / Custom Model Compare

Compare Default vs Custom

Use this after training a custom cluster model.

Custom model metrics

SILHOUETTE

0.4166

Metric comparison

	model_source	saved_model	silhouette
0	Default Model	manual evaluation needed	None
1	Custom Model	None	0.4166

Key Design Features

- Modular architecture
- Separation of concerns
- Reusable components
- Consistent UI across modules

Limitations

- Depends on notebook-trained models
- Not fully production-ready
- Some preprocessing not fully serialized

Conclusion

- Successfully converted notebooks into an app
- Unified multiple ML workflows
- Built a scalable and maintainable system